

Advanced Programming in the Linux Environment

Volume 0

Tools

Marco Bruno

About the author

Marco Bruno

Marco Bruno is a software developer and educator with over twenty years of experience in information technology and professional training. He began his career as a system administrator, working extensively with UNIX and Linux systems, where he developed strong expertise in system-level operations.

Over time, he specialized in C programming on Linux, focusing on system programming and low-level software development. He has taught these subjects through private courses and technical training programs, combining practical experience with a clear and methodical teaching approach. He has also gained significant international working experience, collaborating with companies and professionals across different countries and technical environments. This exposure has contributed to a broad and adaptable perspective on software development practices and system architectures.

Throughout his career, Mr. Bruno has developed software for a wide range of companies, working across multiple domains, including system software, user applications, network programming, and web development. His experience spans several widely used programming languages, including C, Python, and Java.

He combines extensive industry experience with a strong ability to both develop complex software systems and effectively teach their underlying principles.

Preface

This book was created with the goal of bringing order to the Linux manual pages and providing a structured learning path for system programming on Linux.

Linux documentation is vast but often fragmented, and does not follow a clear pedagogical sequence. In addition, many existing books either omit important topics or do not explore them in sufficient depth. *Advanced Programming in the Linux Environment* was written to address this gap by organizing system programming concepts in a logical, progressive order.

The aim of this book is to transform the material found in the Linux manual pages into a coherent and guided learning experience. Topics are introduced in a structured, step-by-step manner, following a natural progression from fundamental concepts to more advanced features of the Linux programming environment.

A new stylistic convention is introduced throughout this book: function names are written without parentheses, and header file names are written without angle brackets. This choice is intended to improve readability and keep the focus on concepts rather than syntax.

The development environment used throughout this book is Kali Linux. This choice was made due to the simplicity of installing tools and libraries through the `apt` package manager, as well as the advantages provided by a Debian-based system during development and experimentation.

Each topic is covered in depth with multiple practical examples, and all code is carefully commented to ensure clarity. The standard structure followed in each section is: explanation of the API, code example, analysis of the code, execution, notes on execution behavior, and further insights.

Special attention is given to depth and completeness. Rather than providing superficial coverage, this book explores the behavior, design, and practical use of Linux system interfaces in detail, helping readers develop a thorough understanding of how Linux systems operate.

The result is a comprehensive text that serves both as a learning guide and as a practical reference, designed to support readers in mastering the complexities of Linux system programming.

Volume 0 — Tools

This introductory chapter presents the essential tools commonly used for software development in a Linux environment, along with an overview of the standards that define software behavior and portability. It is intended to provide readers with the necessary background for the topics covered in the rest of the book; those already familiar with these concepts may choose to skip it.

The chapter covers the compilation process and the use of key development tools such as *gcc*, *make*, and *autoconf*, as well as debugging with *gdb*. It also introduces the source code and how it can be obtained and organized in a Linux system. In addition, it outlines the most widely used software standards at a high level and includes a simple introduction to the GNU General Public License (GPL) and its role in software development and distribution.

Convention used in the book

In the text, function names are written without parentheses (). Although this may initially seem unusual in the UNIX and Linux environment, the author intentionally adopted this convention for several reasons. One of the main motivations is to help the reader become familiar with recognizing system calls and library functions immediately by name, an important skill when studying Linux system programming.

Another stylistic choice differs from the common notation typically used in manuals and documentation. Instead of using angle brackets in paths such as `/proc/<pid>/fd`, the text uses the form `/proc/pid/fd`, where *pid* is written in italics to indicate a variable component of the path. This notation was chosen to improve readability and maintain consistency throughout the text.

For each system call and library function discussed in the text, a footnote is included containing a reference to the corresponding manual page. This allows the reader to quickly access the official documentation and explore additional details, usage information, return values, error conditions, and related functions.

We will use the following format for the code, the function prototypes and the execution of programs.

Code

The `hello.c` program prints a string on the console.

```
1 #include <stdio.h>
2 int main()
3 {
4     printf("Hello, world! \n");
5     return 0;
6 }
```

hello.c

Function prototypes

The `printf`¹ function is defined as follows.

```
#include <stdio.h>

int printf(const char *format, ...);
```

Returns: the number of printed characters

1 See the `printf(3)` manual page

Execution of programs and commands

We can see the manual page of `printf` using the `man2` program.

```
[linux@basis] man 3 printf
```

```
[linux@basis] man -s 3 printf
```

```
[linux@basis] man -S 3 printf
```

```
[linux@basis] man --sections=3 printf
```

The options `-S`, `-s` and `-sections` specify the section to which the manual page belong.

Code analysis

The code analysis can be presented in three different formats:

1. Line-by-line (italicized)

Line 4: Prints the string to standard output.

Line 5: Exits the program successfully

2. Explanatory

At line 4, the string is printed to standard output using `printf`

At line 5, the program terminates successfully using `return`

3. Highlighted

The call

```
4     printf("Hello, world! \n");
```

prints the message to standard output followed by a newline.

² See the `man(1)` manual page

Table of contents

Chapter 0 – Tools

- 0.1 Compilation
- 0.2 The GNU C Compiler
 - 0.2.1 Compiling
 - 0.2.2 Documentation
 - 0.2.3 Warnings and optimization levels
 - 0.2.4 The GNU Linker: `ld`
 - 0.2.5 Executable files
 - 0.2.6 Cross-compiling
- 0.3 The standard library and the headers
- 0.4 GNU Make
- 0.5 GNU Autoconf
- 0.6 The GNU Debugger: `gdb`
- 0.7 The Linux File System
- 0.8 Documentation
 - 0.8.1 Manual pages
 - 0.8.2 `what is`
 - 0.8.3 `info`
 - 0.8.4 `apropos`
 - 0.8.5 The *glibc* documentation
- 0.9 The source code
- 0.10 The Standards
- 0.11 The GNU General Public License
- 0.12 Theory and insights
 - 0.12.1 Byte order
 - 0.12.2 Static and dynamic linking
- List of tables
- List of figures
- Bibliography

0.1 Compilation

The basic tools required for programming are a text editor and a compiler. On Linux systems, many text editors are available, such as `vim`, `nano`, and `gedit`, and in most cases they are already installed. The most commonly used compiler on Linux systems is `gcc`, the GNU C Compiler³. On Debian-based systems, software can be installed and managed using the `apt` package manager.

```
[linux@basis] sudo apt install gcc
...output...
```

Once the compiler is installed and the program has been written using a text editor, it can be compiled with `gcc` in the following way.

```
[linux@basis] gcc -o hello hello.c
[linux@basis]
```

The `-o` option specifies the name of the executable file. After the compilation has terminated we can run the executable.

```
[linux@basis] ./hello
Hello, world!
```

To check the return value of the program, you can use the `echo` command with the `$?`, which prints the *exit value* of the last program executed.

```
[linux@basis] echo $?
0
```

At the end of execution, the program returned a value of 0 to the shell, indicating that it terminated successfully without errors.

If no output file is specified with the `-o` option, `gcc` automatically creates the `a.out` executable file.

```
[linux@basis] gcc hello.c

[linux@basis] ls
a.out  hello  hello.c

[linux@basis] ./a.out
Hello, world!
```

³ Visit the GNU (GNU is Not Unix) official website www.gnu.org

The build process is divided into 4 stages:

Preprocessing

The preprocessor removes comments, expands macros and headers files

Compilation

The compiler generates the assembly code

Assembly

The assembler converts the assembly code into binary machine code by creating the object file

Linking

The linker combines all object files into one executable file and links functions in external libraries to the program code.

Each of these four stages is carried out by a software component (preprocessor, compiler, assembler, and linker), which are often integrated into a single compiler toolchain.

A compiler is a piece of software that takes one or more source code files as input, processes them, and translates them into machine code. In other words, it reads instructions written in a high-level language such as C and converts them into the machine language of the target processor, producing an executable file.

C Source → Compilation → Executable file (binary)

hello.c → gcc → hello

During linking, the program is connected to the libraries that contain the functions it uses. Libraries can be linked in two ways: *static* or *dynamic*. In *static linking*, the code of the library functions is included directly in the executable file. In *dynamic linking*, the library functions are loaded and executed at run time, when the program is running. *Dynamic linking* has the advantage of producing smaller executable files, but it requires the necessary libraries to be present on the system at run time.

The default compiler for the C language on Linux systems is `gcc` (GNU Compiler Collection), while the default linker is `ld`. The `gcc` compiler handles both compilation and linking to libraries, since it internally invokes the necessary tools to complete the build process. The `ld` linker performs the actual task of linking object files and libraries, and it is typically called by `gcc` during compilation unless otherwise specified.

The executable file generated by the compiler is adapted to the underlying operating system or to the one indicated during compilation, in this case Linux. If the same program is compiled on a different operating system, the size of the executable file changes. This is because the compiler compiles the source file based on the operating system on which the executable is to run and therefore tailors the executable to that operating system. To run the same program on two different operating systems, it must be recompiled unless the programming language in which the program was written uses a virtual machine.

The figure below shows the process of compiling a single source file.

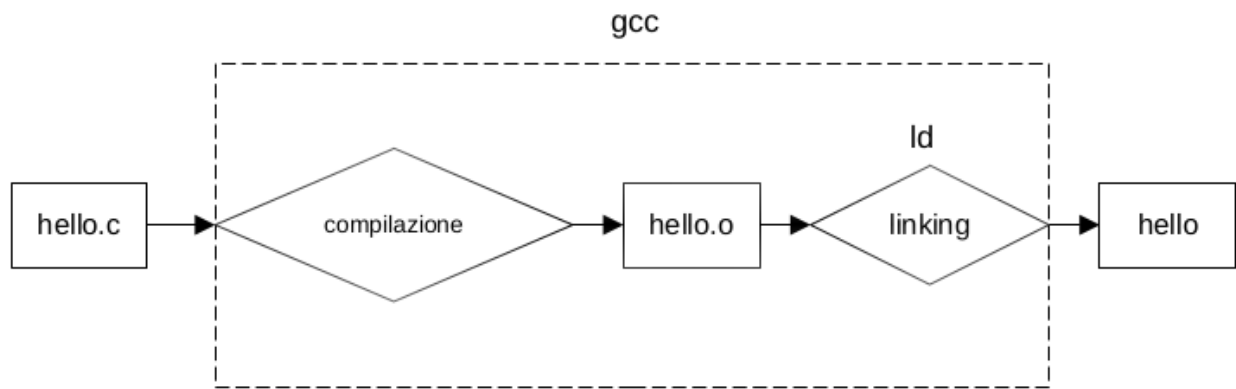


Fig 1. Compilation

The compiler-generated executable file is loadable into the memory and executable by the CPU. On Linux, by convention, executable files have no extension, however, some software uses extensions for clarity.

0.2 The GNU C Compiler

The `gcc` compiler handles all four stages of the build process. It is a powerful compiler that supports a large number of options⁴.

0.2.1 Compiling

To compile a source file, the following syntax is used.

```
[linux@basis] gcc hello.c -o hello
```

```
[linux@basis] ls
```

```
hello
```

`gcc` creates the `hello` executable file. If you omit the `-o` option and the executable name, `gcc` automatically creates the executable file `a.out`.

```
[linux@basis] gcc hello.c
```

```
[linux@basis] ls
```

```
a.out
```

To compile the source file and create the *object file* without linking, use the `-c` option.

```
[linux@basis] gcc -c hello.c
```

The compiler creates the object file `hello.o`. The object file contains information about the symbols (global objects, functions, etc..) that must be loaded

To link the object file and create the *binary file*, we pass the object file to `gcc` and specify the executable file with `-o`.

```
[linux@basis] gcc hello.o -o hello
```

`gcc` creates the `hello` executable file. If the filename is not specified and the `-o` option is omitted, `gcc` automatically creates the executable file `a.out`.

To save the files created by each step, we can use the `-save-temps` option.

```
[linux@basis] gcc -save-temps hello.c
```

```
[linux@basis] ls
```

```
hello.i hello.o hello.s a.out hello.c
```

⁴ See the `gcc(1)` manual page

The following table lists the extensions of the files saved with `-save-temps`

<code>.i</code>	preprocessed file
<code>.o</code>	object file
<code>.s</code>	assembly file
<code>.out</code>	executable file
<code>.c</code>	c-source file

Tab 1. Files extensions created by `-save-temps`

To create the file containing the assembly instructions we can use the `-S` option.

```
[linux@basis] gcc -S hello.c
```

`gcc` creates the file `hello.s`. It is possible to compile the assembly file to create the executable in the following way

```
[linux@basis] gcc hello.s -o hello
```

If we want to stop the build process after the preprocessing stage we can use the `-E` option as follows

```
[linux@basis] gcc -E hello.c -o hello.cpp
```

If we print the contents of `hello.cpp` using `cat`⁵, we can see that the contents of the `stdio.h` header file have been inserted into the source code.

We can specify the language of the code we want to compile.

```
[linux@basis] gcc -x cpp-output hello.cpp -o hello
```

The `gcc -x` option allows you to specify the language. In this case `cpp-output` represents a C file generated by the preprocessor (`cpp` stands for C PreProcessing)⁶.

To compile multiple source files belonging to a single project, the syntax is as follows

```
[linux@basis] gcc hello.c hello.c hello.h -o hello
```

⁵ `cat` is a program used to display the contents of text files, see the `cat(1)` manual page

⁶ Programs written in C++ have the extension `.cpp`, it is good to distinguish these programs from the C code generated by the preprocessor

With the `-I` option of `gcc` we can specify an alternative directory for header files⁷. The following `hello.c` program, which uses the `hello.h` header file, can be compiled by specifying the directory containing the header file.

```
#include "hello.h"
#include <stdio.h>

void main()
{
    printf ("%s", GREETING);
}
```

hello.c

```
#define GREETING "Hello, world!"
```

hello.h

```
[linux@basis] gcc -o hello hello.c -I.
```

```
[linux@basis] ./hello
Hello, world!
```

The `-I` option followed by `.` tells `gcc` to search for header files in the current directory in addition to `/usr/include`.

With the `-L` option we can specify a directory for libraries and with the `-l` option we can link to a specific library.

The following table lists some of the most common `gcc` options⁸.

<code>-o FILE</code>	Specifies the output file
<code>-c</code>	Build without linking
<code>-I dir</code>	Search the header files in the directory <code>dir</code>
<code>-L dir</code>	Search the libraries in the directory <code>dir</code>
<code>-static</code>	Link to static library
<code>-llib</code>	Link to the <code>lib</code> library
<code>-g</code>	Includes debug information in the executable file
<code>-ggdb</code>	Includes debug information that <code>gdb</code> can use

⁷ By default, the directory in which `gcc` looks for headers is `/usr/include`

⁸ For the complete list, consult the `gcc (1)` manual page and the documentation listed in the bibliography

<code>-O</code>	Optimize the compiled code
<code>-ON</code>	Specifies an optimization level (0-3)
<code>-pedantic</code>	Issue all the warnings of the ANSI/ISO C standard
<code>-w</code>	Suppress all warnings
<code>-Wall</code>	Issues all or most of the warnings that <code>gcc</code> can issue
<code>-Werror</code>	Convert all warnings to errors
<code>-MM</code>	Create a make-compatible dependency list
<code>-v</code>	Display the commands used in each step of the build
<code>-h</code>	Display a summary help

Tab 2. `gcc` options

The following figure shows each step of compilation process with `gcc`, except for preprocessing.

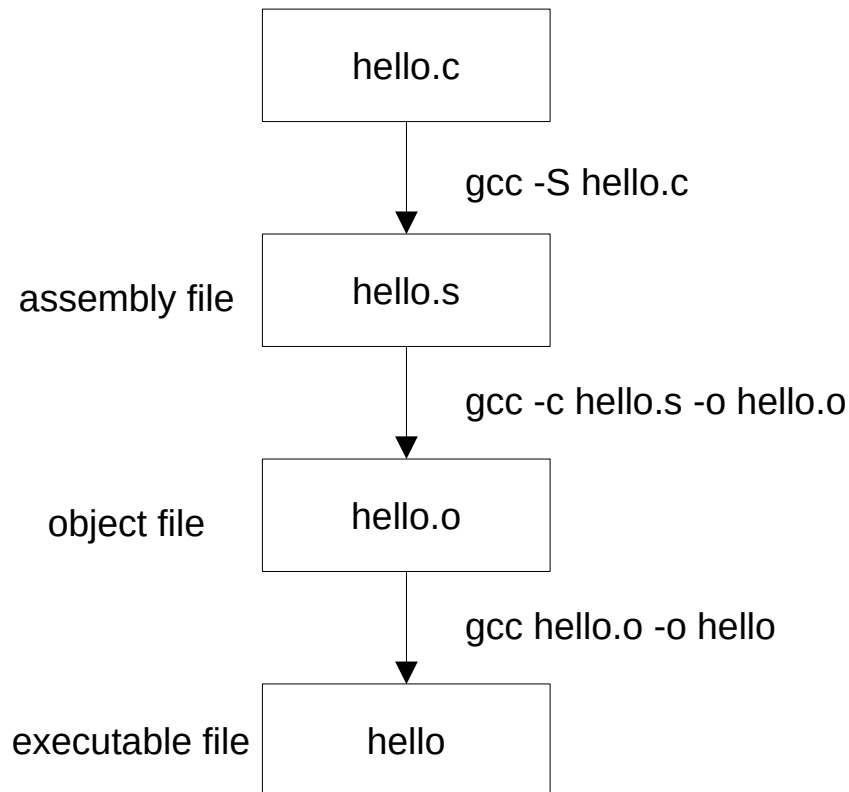


Fig 2. Compilation steps with `gcc`

0.2.2 Documentation

On Debian-based systems, the documentation can be installed using `apt`.

```
[linux@basis] sudo apt install gcc-doc  
...output...
```

Once the package containing the documentation has been installed, we can view the manual page

```
[linux@basis] man gcc  
...man page...
```

In addition to the documentation package, GNU also provides a manual that can be downloaded from the official website⁹.

⁹ See the 'Using the GNU Compiler Collection' link <https://gcc.gnu.org/onlinedocs/>

0.2.3 Warning and optimization levels

During compilation, we can specify the level of strictness for warning messages. The `-Wall` option enables most of the warnings supported by the compiler.

The `hello.c` program prints a string on the console.

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    int x;
    printf ("Hello, world!");
    return;
}
```

hello.c Prints 'Hello, world!'

Let's compile the code

```
[linux@basis] gcc -o hello hello.c
```

```
[linux@basis]
```

The compilation succeeded. Let's compile passing `-Wall`

```
[linux@basis] gcc -Wall -o hello hello.c
hello.c:3:6: warning: return type of 'main' is not 'int' [-Wmain]
   3 | void main(int argc, char *argv[])
     |      ^~~~
hello.c: In function 'main':
hello.c:5:13: warning: unused variable 'x' [-Wunused-variable]
   5 |         int x;
     |         ^
[linux@basis]
```

The compiler displays two warnings:

1. The return type of the `main` is not `int` (we did not specify a value to `return`)
2. The `x` variable is not used (we did not initialize it nor use it)

The `--Wall` option evaluates many control flags¹⁰ of the code.

¹⁰ For a complete list see the `gcc (1)` manual page

Another option for warning analysis is `-Wextra` which enables some checks not included in `-Wall` such as for example the comparison of a pointer with the value 0 using the operators `<`, `<=`, `>`, `>=`.

The `hellopointer.c` program compares a pointer with 0

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    int x = 4;
    int *y;
    y = &x;
    if (y > 0)
        printf ("Hello, world!");
    return 0;
}
```

hellopointer.c Compare a pointer with the value 0

Let's compile with `-Wall`

```
[linux@basis] gcc -Wall -o hellopointer hellopointer.c
```

```
[linux@basis]
```

Now compile with `-Wextra`

```
[linux@basis] gcc -Wextra -o hellopointer hellopointer.c
hellopointer.c: In function 'main':
hellopointer.c:8:15: warning: ordered comparison of pointer with integer ze
ro [-Wextra]
     8 |         if (y > 0)
       |         |
```

```
[linux@basis]
```

The `--Wextra` option detects the comparison between the pointer and the value 0.

The `-g` option causes `gcc` to include debugging information in the executable file. This information is entered in the native format of the system on which the compilation is performed. The `gdb` program can use this information. If you are sure you are using `gdb` as a debugging tool you can use the `-ggdb` option which includes debug information for use with `gdb`. Among the debug information is the source code of the program.

The `hellopointer.c` program compiled with the `-g` option and passed to `gdb` for debugging.

```
[linux@basis] gcc -Wall -g -o hellopointer hellopointer.c
```

```
[linux@basis] gdb -q ./hellopointer
```

```
Reading symbols from ./hellopointer...
```

```
(gdb) list
```

```
1      #include <stdio.h>
2
3      int main(int argc, char *argv[])
4      {
5          int x = 4;
6          int *y;
7          y = &x;
8          if (y > 0)
9              printf ("Hello, world!");
10         return 0;
(gdb)
```

With the `list` command of `gdb` we can view the source code that has been inserted into the executable file thanks to the `-g` option passed to `gcc`.

`gcc` provides several debugging options, including options that specify the amount of debugging information to include in the executable file¹¹. Levels range from 0 (no debug information) to 3 (debug detailed information), the default level is 2. The level of information to be entered can be specified by placing the relative number alongside the `-g` option (e.g. `-g3`). Obviously the size of the executable file changes according to the selected level.

```
[linux@basis] gcc -Wall -g1 -o hellopointer hellopointer.c
```

```
[linux@basis] ls -l hellopointer
```

```
-rwxrwxr-x 1 basis basis 16920 Oct 1 20:38 hellopointer
```

```
[linux@basis] gcc -Wall -g3 -o hellopointer hellopointer.c
```

```
[linux@basis] ls -l hello
```

```
-rwxrwxr-x 1 basis basis 44688 Oct 1 20:39 hellopointer
```

The `-O` option allows to optimize the code. In this way the goal of the compiler is no longer to reduce the cost of compiling time but to improve the execution performance of the program and to reduce the size of the executable file.

The `-O` option accepts optimization levels 0 to 3. Levels are inclusive in the sense that level 2 contains all optimizations of level 1, and so on.

Optimization options are an advanced topic in programming and include elements that are beyond the scope of this book. An example is shown below for information.

The following `helloinc.c` program increments the variable `sum` on each iteration

```
#include <stdio.h>
```

¹¹ See the `gcc(1)` manual page

```
int main(int argc, char *argv[])
{
    int x = 1;
    int sum;

    for (int i=0; i<1000000000; i++) {
        sum += i;
        sum += x;
    }
    return 0;
}
```

helloinc.c Increments the variable on each iteration

Let's compile without optimization and execute using `time`¹² to look at the execution time

```
[linux@basis] gcc -o helloinc helloinc.c
```

```
[linux@basis] time ./helloinc
```

```
real  2.42s
user  2.42s
sys    0.00s
cpu    99%
```

Let's compile passing the optimization level 3

```
[linux@basis] gcc -O3 -o helloinc helloinc.c
```

```
[linux@basis] time ./helloinc
```

```
real   0.00s
user   0.00s
sys    0.00s
cpu    80%
```

Execution time and CPU usage decreased significantly after enabling compiler optimizations. However, this test is not entirely reliable, since the results may be influenced by factors such as system load, cache usage, background processes, compiler optimizations, and variations in hardware resource allocation during execution. In some cases, the compiler may also optimize away parts of the code that are not essential to the program output. Readers are encouraged to test programs compiled with different optimization levels in various contexts and compare their execution times using the `time` command.

¹² `time` calculates the execution time of a program, see the `time(1POSIX)` manual page

0.2.4 The GNU Linker: `ld`

The dynamic linker (`/usr/bin/ld.so` or `/lib/ld-linux.so.2`) is responsible for locating and loading the shared objects (shared libraries) required by a program, preparing it for execution, and then starting it. Linux binaries typically use dynamic linking (linking at run time) unless the `-static` option is specified during compilation.

The program `ld.so` handles `a.out` binaries, an older binary format. The program `ld-linux.so*` (such as `/lib/ld-linux.so.1` for `libc5` and `/lib/ld-linux.so.2` for `glibc2`) handles binaries in the more modern ELF format.

Usually, `gcc` automatically invokes the dynamic linker to link the executable with the shared libraries, but linking can also be performed separately.

To compile a program without linking we can use the `-c` option of `gcc`

```
[linux@basis] gcc -c hello.c -o hello.o
```

The resulting file (`hello.o`) is an object file containing the machine code without any entry point to shared libraries.

To link this file to the required shared libraries, we need to be familiar with the following options of the `ld` program (the GNU linker).¹³

<code>-lc</code>	Link to the C standard library
<code>-dynamic-linker</code>	Specify the dynamic linker
<code>-e main</code>	Specify the entry symbol
<code>crt1.o, crt1.o, crtn.o</code>	C runtime startup code

Let's link the `hello.o` file

```
[linux@basis] ld hello.o -o hello -lc -dynamic-linker /lib64/ld-linux-x86-64.so.2 -e main
/usr/lib/x86_64-linux-gnu/crt1.o /usr/lib/x86_64-linux-gnu/crti.o /usr/lib/x86_64-linux-gnu/crtn.o
```

And execute

```
[linux@basis] ./hello
Hello, world!
```

¹³ See the `ld(1)` manual page

0.2.5 Executable files

On Linux, the standard format for executable files is ELF (*Executable and Linkable Format*). Using the `file`¹⁴ program, you can determine the type of a file.

```
[linux@basis] file hello.c
hello.c: C source, ASCII text
```

```
[linux@basis] file hello
hello: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter
/lib64/ld-linux-x86-64.so.2, BuildID[sha1]=dfb6ddef98a884e5d6caf63dc66e9d014e8ad9ea, for
GNU/Linux 3.2.0, not stripped
```

The `hello.c` file is a text file. The `hello` file is an executable file in ELF format. The following are the values returned by the `file` command and their meanings.

64-bit

The type of architecture. Processor registers, memory addresses and data are 64 bits in size

LSB

The executable is *Least Significant Byte First*, i.e. the least significant byte is placed first

pie

The program is a position-independent executable (PIE), meaning that its code, once loaded into memory, can execute correctly regardless of the memory address at which it is loaded. PIE programs do not require a specific memory address to be loaded in order to run, anywhere in memory they are loaded they can be executed

86-64

It indicates the type of processor architecture for which the program was compiled. *x86* refers to the processor family, while *64* indicates that the architecture uses 64-bit registers.

version 1 (SYSV)

Version of the ELF standard

dynamically linked

Libraries are dynamically linked to the program

interpreter

The *dynamic linker* used by the program (`/lib64/ld-linux-x86-64.so.2`)

BuildID

Identifier of the build session that produced the executable file

for GNU/Linux 3.2.0

The operating system for which the program was compiled. 3.2.0 indicates the minimum kernel version

¹⁴ See the `file(1)` manual page

not stripped

The executable file has not been optimized using the `strip` command¹⁵

¹⁵ See the `strip(1)` manual page

0.2.6 Cross compiling

To compile a program for a processor different from the one you are currently using, you need to install the appropriate compiler. Compiling for a target platform different from the host system is called *cross-compiling*.

For example, to compile a C program for a 32-bit or 64-bit ARM processor, you must install the corresponding cross-compilers.

```
[linux@basis] sudo apt install gcc-arm-linux-gnueabi gcc-aarch64-linux-gnu
...output...
```

Once the compiler is installed, the program can be compiled.

```
[linux@basis] arm-linux-gnueabi-gcc hello.c -o hello-arm -static
[linux@basis] file hello-arm
hello-arm: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), statically linked,
BuildID[sha1]=f2678f1ce26ac6ca1dbc5fc7cfd13dc57d67e673, for GNU/Linux 3.2.0, not stripped
```

32-bit	32-bit architecture
ARM	The processor for which the program was compiled
EABI5	<i>Embedded-Application Binary Interface</i> . The executable file interface for embedded operating systems ¹⁶ .
static linked	Libraries are statically linked

To compile for 64-bit ARM

```
[linux@basis] aarch64-linux-gnu-gcc hello.c -o hello-arm64 -static
[linux@basis] file hello-arm64
hello-arm64: ELF 64-bit LSB executable, ARM aarch64, version 1 (GNU/Linux), statically linked,
BuildID[sha1]=56ac82782fd30fbfbf729f2eae1c2359e8a6c162, for GNU/Linux 3.7.0, not stripped
```

64-bit	64-bit architecture
ARM aarch64	64-bit ARM processor

The executable files generated will run respectively on a Linux operating system (kernel $\geq$ 3.2.0 and 3.7.0) installed on a 32 and 64 bit ARM processor.

¹⁶ Embedded systems are those systems that have a specific function, e.g. mobile phones, televisions, video game consoles, industrial machine controllers, digital clocks, etc..

0.3 Standard library and header files

The standard C library includes many header files, which on Linux are typically located in `/usr/include`.

On Debian-based systems, packages that provide these headers usually end with the `-dev` suffix (for example, `libc6-dev`). If some headers are missing, they can be installed by installing the corresponding library development package using the `-dev` suffix. For instance, to use features provided by the MP3 library (`libmpg123`), you need to install the `libmpg123-dev` package, which will place the required headers in the appropriate directory.

Development packages may also include shared libraries.

Libraries are typically located in directories such as `/lib`, `/usr/lib`, `/lib64`, and `/usr/lib64`. Additional or locally installed libraries are usually placed in `/usr/local/lib`.

On Linux, static libraries use the `.a` extension, while shared libraries use the `.so` (shared object) extension.

Shared libraries follow the naming convention `library_name-version.so`, for example `libc-2.33.so`. Symbolic links to these libraries typically use the format `library_name.so.version`, such as `libc.so.6`.

To determine which shared libraries an executable depends on, you can use the `ldd` command.

```
[linux@basis] ldd ./HelloWorld
linux-vdso.so.1 (0x00007ffd22579000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f1b6ecad000)
/lib64/ld-linux-x86-64.so.2 (0x00007f1b6eecd000)

[linux@basis] ldd /bin/ls
linux-vdso.so.1 (0x00007ffcceba7000)
libselinux.so.1 => /lib/x86_64-linux-gnu/libselinux.so.1 (0x00007f2dd6083000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f2dd5eba000)
libpcre2-8.so.0 => /lib/x86_64-linux-gnu/libpcre2-8.so.0 (0x00007f2dd5e22000)
libdl.so.2 => /lib/x86_64-linux-gnu/libdl.so.2 (0x00007f2dd5e1b000)
/lib64/ld-linux-x86-64.so.2 (0x00007f2dd6127000)
libpthread.so.0 => /lib/x86_64-linux-gnu/libpthread.so.0 (0x00007f2dd5dfa000)
```

Library packages usually have the `lib` prefix, for example `libncurses5`.

The program that manages the dynamic linker's library cache is `ldconfig`¹⁷.

```
[linux@basis] sudo ldconfig
```

The standard C library on Linux is the GNU C Library, known as `glibc` (GNU C Library). At the time of writing, the released version is 2.33. `glibc` provides standard C functionality as well as additional extensions¹⁸.

¹⁷ See the `ldconfig(8)` manual page

¹⁸ See <https://www.gnu.org/software/libc/> and the *Bibliography*

0.4 GNU Make

`make`¹⁹ is a program that allows you to automate the compilation of multiple source files by defining rules in a file called a `Makefile`. The `Makefile` contains rules that specify what to compile and how. A rule consists of:

1. A target - what `make` tries to create (the final executable program)
2. A list of one or more dependencies (files required to compile the target)
3. A list of commands to execute to create the target

When `make` is run it looks for a file called `GNUmakefile`, `makefile` or `Makefile` (in that order) in the directory it was run from.

The rules have the following form:

```
target : dependency dependency [...]  
        command  
        command  
        [...]
```

The following example shows a source file, a header file and a `Makefile`.

```
#include "hello.h"  
  
int main()  
{  
    printf ("\nHello, World");  
    return 0;  
}
```

hello.c

```
#include "hello.h"
```

hello.h

¹⁹ See the `make(1)` manual page

```
1 hello : hello.o
2         gcc -o hello hello.o

3 hello.o : hello.c hello.h
4         gcc -c hello.c

5 clean :
6         rm hello *.o

7 install :
8         cp hello /usr/local/bin
```

Makefile

- Line 1. Specifies the target executable file to create (`hello`) and, after the `:`, the dependency used to build it, i.e. `hello.o`*
- Line 2. The compilation command (`gcc`) used to generate the executable*
- Line 3. Specifies the object file to create (`hello.o`), which is later used to build the executable, and the source files required to build it (`hello.c` and `hello.h`)*
- Line 4. The command used to compile the source file into an object file*
- Line 5. The `clean` target deletes files generated by a previous build (the `hello` executable and object files with the `.o` extension). It is used when recompilation is needed*
- Line 6. The command that removes the files generated by compilation (`rm`)*
- Line 7. After building the executable, the `install` target is defined to install it in a system directory containing executable programs (`/usr/local/bin`)*
- Line 8. The `cp` command used to copy the executable file into a system path*

Below is the execution of `make`, which reads the `Makefile` and compiles the program accordingly.

```
[linux@basis] make
cc -c -o hello.o hello.c
gcc -o hello hello.o
```

`make` have created the executable file `hello` and the object file `hello.o`.
Below is the execution of `make clean`

```
[linux@basis] make clean

rm hello *.o
```

The files `hello.o` and `hello` have been deleted.

We run `make` again to recreate `hello` and `hello.o`.

```
[linux@basis] make
```

```
cc -c -o hello.o hello.c  
gcc -o hello hello.o
```

We install the `hello` executable file in the system directory `/usr/local/bin`

```
[linux@basis] sudo make install
```

```
cp hello /usr/local/bin
```

`make` is a very powerful tool and one of the most widely used in the development of programs on Linux systems. The set of rules that can be defined is quite extensive, and in addition to these, it is possible to define variables to configure various parameters, such as the target hardware architecture of the executable file, compilation options, and much more.

The `make(1)` manual page lists all available options; for complete documentation, you can also consult the official website listed in the bibliography.

0.5 GNU Autoconf

`autoconf`²⁰ is a tool used to automate source code configuration. It generates a configuration script from a template file. On Debian-based systems, `autoconf` can be installed by running the following command.

```
[linux@basis] sudo apt install autoconf
...output...
```

To use `make` to compile the source code, it is necessary to generate a `Makefile`. To generate it, you can use the `automake`²¹ program, which takes as input a file named `Makefile.am` and produces a file named `Makefile.in`. This file is then read by the `configure` script generated by `autoconf`, in order to create the final `Makefile`.

To proceed with the compilation, it is necessary to create two files: `configure.ac` and `Makefile.am`.

Below is an example showing how to compile the file `hello.c`.

1. We create the file `configure.ac`

```
AC_INIT([hello], [1.0], [io@example.com])
AM_INIT_AUTOMAKE
AC_PROG_CC
AC_CONFIG_FILES([Makefile])
AC_OUTPUT
```

The directives used

<code>AC_INIT</code>	→ Specifies the name of the program, the version, the author
<code>AM_INIT_AUTOMAKE</code>	→ Creates the configuration for the use of <code>automake</code>
<code>AC_PROG_CC</code>	→ Specifies the compiler (<code>gcc</code>)
<code>AC_CONFIG_FILES</code>	→ Specifies the configuration file that will be generated (<code>Makefile</code>)
<code>AC_OUTPUT</code>	→ Specifies the file that will be generated (<code>configure</code>)

2. We create the file `Makefile.am`

```
AUTOMAKE_OPTIONS = foreign
bin_PROGRAMS = hello
hello_SOURCES = hello.c
```

The directives used

<code>AUTOMAKE_OPTIONS</code>	→ Automake options. <code>foreign</code> defines a level of narrowness regarding the requirements of the compilation ²²
-------------------------------	--

²⁰ See the `autoconf(1)` manual page, the official website is listed in the bibliography

²¹ See the `automake(1)` manual page

²² For more details on this topic see https://www.gnu.org/software/automake/manual/html_node/Strictness.html

bin_PROGRAMS → The name of the executable file that will be generated
hello_SOURCES → Source files to compile

Let's take a look at the contents of the directory.

```
[linux@basis] ls  
configure.ac hello.c Makefile.am
```

These instructions are actually macros, i.e. functions that are expanded into one or more underlying instructions. To work with these macros, a macro processor is required, i.e. software that reads the macros and executes the corresponding instructions. On Linux, the macro processor used by `autoconf` is `m4`²³.

For macro compatibility reasons, before running `autoconf`, it is necessary to run `aclocal`²⁴. This program ensures that all macros required for the build process are available²⁵. To use the `AM_INIT_AUTOMAKE` macro, the file `aclocal.m4` must be generated using `aclocal`.

3. Let's execute `aclocal`

```
[linux@basis] aclocal
```

Now we have the following files

```
[linux@basis] ls  
aclocal.m4 autom4te.cache configure.ac hello.c Makefile.am
```

4. Let's create the file `configure` using `autoconf`

```
[linux@basis] autoconf
```

5. Let's create the `makefile`

```
[linux@basis] automake --add-missing  
...output...
```

Now we have also the files `configure` e `Makefile.in`

```
[linux@basis] ls  
aclocal.m4 autom4te.cache compile configure configure.ac depcomp hello.c install-sh  
Makefile.am Makefile.in missing
```

6. Let's execute `configure` to generate the `Makefile`

23 For more details on the macro processor `m4` see <https://www.gnu.org/software/m4/manual/>

24 See the `aclocal(1)` manual page

25 For more details on `aclocal` see https://www.gnu.org/software/gettext/manual/html_node/aclocal.html

```
[linux@basis] ./configure  
...output...
```

The Makefile has been created

```
[linux@basis] ls  
aclocal.m4 autom4te.cache compile config.log config.status configure configure.ac depcomp  
hello.c install-sh Makefile Makefile.am Makefile.in missing
```

7. Let's compile it using make

```
[linux@basis] make  
...output...
```

make creates the hello.o object file and the hello executable file.

```
[linux@basis] ls  
aclocal.m4 autom4te.cache compile config.log config.status configure configure.ac depcomp  
hello hello.c hello.o install-sh Makefile Makefile.am Makefile.in missing
```

The following figure shows the process.

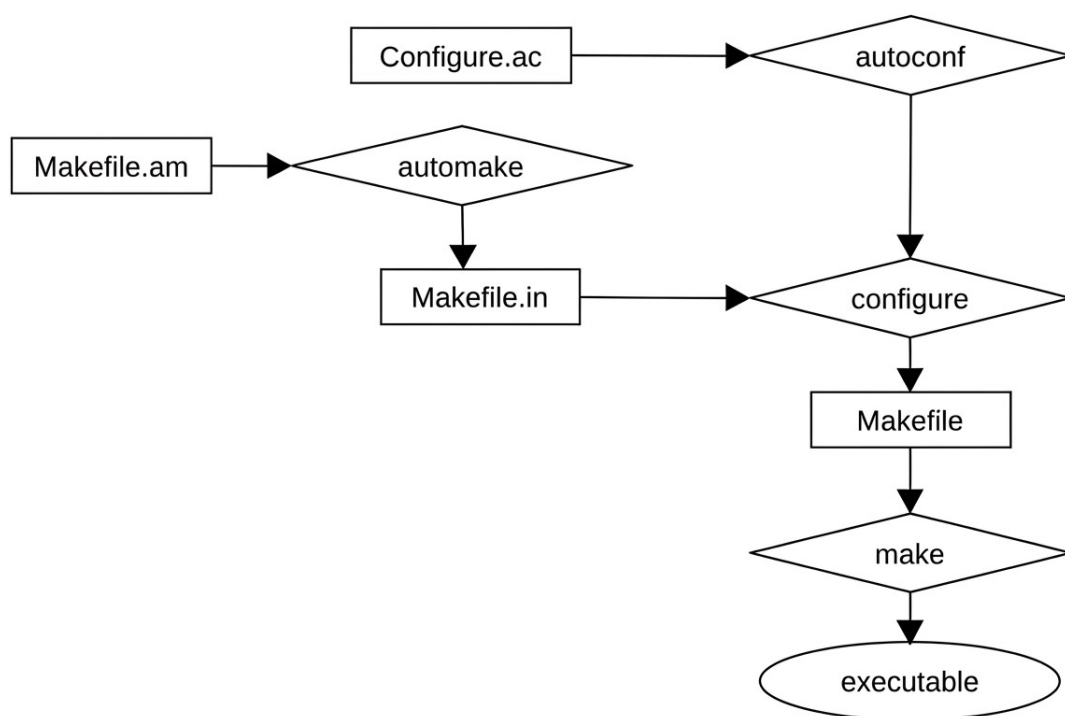


Fig 3. Steps of the compilation process with autoconf

0.6 The GNU Debugger: `gdb`

The GNU Debugger (`gdb`) is the default debugging tool on Linux. It operates in an interactive mode, using a prompt that accepts commands and displays their output. This paragraph serves only as an introduction to the tool.

To install `gdb` on debian-based systems run the following command.

```
[linux@basis] sudo apt install gdb
```

To run it, pass the program you want to debug as a command-line argument

```
[linux@basis] gdb ./hello
GNU gdb (Debian 10.1-2) 10.1.90.20210103-git
Copyright (C) 2021 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
  <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...

Reading symbols from ./hello...
(gdb)
```

After the initial banner (can be removed with the `-q` option) `gdb` reads the symbol table²⁶ of the executable file and displays the prompt.

We can insert debugging information into compile-time programs with the option `-g` of `gcc`.

```
[linux@basis] gcc hello.c -g -o hello
```

Once included, they can be viewed and used. For example, the `list` command in `gdb` displays the source code that is included in the debugging information of the executable.

```
[linux@basis] gdb -q ./hello
Reading symbols from ./hello...
(gdb)
(gdb) break main
Breakpoint 1 at 0x113d: file hello.c, line 5.
```

²⁶ The *symbol table* is the list of the functions used in the program

(gdb) **start**

Temporary breakpoint 2 at 0x113d: file hello.c, line 5.

Starting program: /home/basis/hello

Breakpoint 1, main () at hello.c:5

5 printf ("\nHello, World");

(gdb) **list**

1 #include "hello.h"

2

3 int main()

4 {

5 printf ("\nHello, World");

6 return 0;

7 }

(gdb) **disassemble**

Dump of assembler code for function main:

```
0x000055555555139 <+0>:   push  %rbp
0x00005555555513a <+1>:   mov   %rsp,%rbp
=> 0x00005555555513d <+4>:   lea   0xec0(%rip),%rax    # 0x555555556004
0x000055555555144 <+11>:  mov   %rax,%rdi
0x000055555555147 <+14>:  mov   $0x0,%eax
0x00005555555514c <+19>:  call  0x55555555030 <printf@plt>
0x000055555555151 <+24>:  mov   $0x0,%eax
0x000055555555156 <+29>:  pop   %rbp
0x000055555555157 <+30>:  ret
```

End of assembler dump.

(gdb) **set disassembly-flavor intel**

(gdb) **disassemble**

Dump of assembler code for function main:

```
0x000055555555139 <+0>:   push  rbp
0x00005555555513a <+1>:   mov   rbp,rsb
=> 0x00005555555513d <+4>:   lea   rax,[rip+0xec0]    # 0x555555556004
0x000055555555144 <+11>:  mov   rdi,rax
0x000055555555147 <+14>:  mov   eax,0x0
0x00005555555514c <+19>:  call  0x55555555030 <printf@plt>
0x000055555555151 <+24>:  mov   eax,0x0
0x000055555555156 <+29>:  pop   rbp
0x000055555555157 <+30>:  ret
```

End of assembler dump.

(gdb) **next**

6 return 0;

(gdb) **cont**

Continuing.

Hello, World[Inferior 1 (process 219752) exited normally]

(gdb) **quit**

[[linux@basis](#)]

gdb provides many commands for memory analysis, variable inspection, and other operations²⁷. The following table lists some gdb commands²⁸

backtrace bt	Displays the content of the stack ²⁹
break	Set a break point
continue cont	Continue the execution until the next break point
disassemble	Displays the assembly code
info register i r	Displays the content of a cpu register
list	Displays the source code
nexti	Executes the next instruction
quit	Quit from the debugger
run	Start the exeution of the program
set	Activatea a setting
step s	Continues execution until the next line of source code
x	Examine the contents of one or more memory cells

Tab 3. gdb commands

We can install the documentation using apt

```
[linux@basis] sudo apt install gdb-doc  
...output...
```

The `-q` option allows you to avoid displaying the initial banner during startup.

The `--args` option allows you to pass command-line arguments to the program you want to debug.

We can use the following `hello2.c` program to have a debugging session with gdb

```
#include <stdio.h>  
#include <stdlib.h>  
  
int main (int argc, char *argv[])  
{  
    int x;  
    x = atoi(argv[1]);  
}
```

27 See the `gdb(1)` manual page and the links in the bibliography

28 For a complete list of the gdb features see the `gdb(1)` manual page

29 The `backtrace` command also has the short form `bt`, the same for the other listed commands

```

printf ("%d", x);
return 0;
}

```

hello2.c Prints the passed number on the console

```

[linux@basis] gdb -q --args hello2 4
Reading symbols from hello2...
(No debugging symbols found in hello)
(gdb) break main
Breakpoint 1 at 0x114d
(gdb) set disassembly-flavor intel
(gdb) disassemble main
Dump of assembler code for function main:
0x0000000000001149 <+0>:    push    rbp
0x000000000000114a <+1>:    mov     rbp, rsp
0x000000000000114d <+4>:    sub     rsp, 0x20
0x0000000000001151 <+8>:    mov     DWORD PTR [rbp-0x14], edi
0x0000000000001154 <+11>:   mov     QWORD PTR [rbp-0x20], rsi
0x0000000000001158 <+15>:   mov     rax, QWORD PTR [rbp-0x20]
0x000000000000115c <+19>:   add     rax, 0x8
0x0000000000001160 <+23>:   mov     rax, QWORD PTR [rax]
0x0000000000001163 <+26>:   mov     rdi, rax
0x0000000000001166 <+29>:   call    0x1040 <atoi@plt>
0x000000000000116b <+34>:   mov     DWORD PTR [rbp-0x4], eax
0x000000000000116e <+37>:   mov     eax, DWORD PTR [rbp-0x4]
0x0000000000001171 <+40>:   mov     esi, eax
0x0000000000001173 <+42>:   lea     rax, [rip+0xe8a]    # 0x2004
0x000000000000117a <+49>:   mov     rdi, rax
0x000000000000117d <+52>:   mov     eax, 0x0
0x0000000000001182 <+57>:   call    0x1030 <printf@plt>
0x0000000000001187 <+62>:   mov     eax, 0x0
0x000000000000118c <+67>:   leave
0x000000000000118d <+68>:   ret
End of assembler dump.
(gdb) run
Starting program: /home/basis/hello2 4

Breakpoint 1, 0x000055555555114d in main ()
(gdb) cont
Continuing.
4[Inferior 1 (process 7081) exited normally]
(gdb) quit

```

The `break main` command sets a break point at the beginning of the main function. The `set disassembly-flavor intel` command sets the intel syntax for assembly language. The `disassemble main` command displays the assembly code of the main function. The `run` command starts the execution of the program which stops at the break point set before. The `cont` or `continue` command continues program execution. The `quit` command exits the debugger.

The output of the `disassemble` command shows, in the first column, the memory address of each assembly instruction. The second column indicates the offset in bytes from the first instruction, while the third column contains the corresponding assembly instruction. The differences in the second column between successive entries represent the size of each instruction; for example, the first instruction (`push rbp`) is 1 byte long, as indicated by the next entry showing `<+1>`.

The arrow in the first column marks the current instruction, i.e. the exact point where the program execution is located.

Intel syntax for disassembly is generally more readable than AT&T syntax, as it does not use symbols such as `%`.

0.7 The Linux File System

The *Linux File System* is organized according to a tree structure starting from the root directory represented by the character `/`. The root directory (`/`) is the starting point of the filesystem. Not to be confused with the user `root` which is the superuser of the system and with the directory `/root` that is the home directory of the `root` user. Below is an overview of the Linux filesystem.

In directory `/` the following directories are present:

<code>boot</code>	→ Boot files of the system
<code>dev</code>	→ Device files (audio, video, hard drives, etc..)
<code>etc</code>	→ Configuration files
<code>home</code>	→ Home directories of the users
<code>lost+found</code>	→ Not needed files
<code>media</code>	→ Mount point for removable devices
<code>mnt</code>	→ Mount point for temporary file systems
<code>opt</code>	→ Optional software
<code>proc</code>	→ Running processes on the system (fs <code>proc</code> ³⁰)
<code>root</code>	→ Home directory of the root user (superuser)
<code>run</code>	→ Temporary files of the running processes (lock, logs, etc..) (fs <code>tmpfs</code>)
<code>srv</code>	→ Files of the server services provided by the system (FTP, WWW)
<code>sys</code>	→ Information and kernel interface (fs <code>sysfs</code>)
<code>tmp</code>	→ Temporary files
<code>usr</code>	→ Applications and programs of the users
<code>var</code>	→ Files created by the programs (logs, caches, etc..)

In the `/` folder the following links are present:

<code>bin</code>	→ <code>/usr/bin</code>	→ Link to the executable programs
<code>lib</code>	→ <code>/usr/lib</code>	→ Link to the libraries
<code>lib32</code>	→ <code>/usr/lib32</code>	→ Link to the libraries for 32-bit systems
<code>lib64</code>	→ <code>/usr/lib64</code>	→ Link to the libraries for 64-bit systems
<code>sbin</code>	→ <code>/usr/sbin</code>	→ Link to the executable programs of the superuser (<code>root</code>)

And the files:

<code>initrd.img</code>	→ File system image to mount as RAM disk during the boot phase
<code>vmlinuz</code>	→ Kernel Linux

Some of the most commonly used directories are:

<code>/proc/sys</code>	→ system configuration
<code>/usr/bin</code>	→ executable programs
<code>/usr/lib</code>	→ libraries
<code>/usr/local</code>	→ additional programs and configurations
<code>/usr/sbin</code>	→ executable programs of the superuser

³⁰ The filesystem type. The `proc` directory is attached to a `proc` filesystem, the `run` directory to a `tmpfs` filesystem and the `sys` directory to a `sysfs` filesystem

<code>/usr/share</code>	→ files shared by programs
<code>/usr/share/doc</code>	→ documentation
<code>/var/log</code>	→ system logs
<code>/var/run</code>	→ files and locks of the running processes

The default filesystem type used by Linux systems is `ext4`. For a complete list of Linux filesystem types, see the `filesystems(5)` manual page.

With the `df` program, we can view all currently mounted filesystems.

```
[linux@basis] df -a
...output...
```

A final consideration concerns the swap filesystem, which is used as virtual memory to increase the amount of available memory on a system. The swap space is a file or partition located on disk that is used by processes when physical memory is insufficient, helping to improve overall system performance. To view the partition where swap is located and its size, you can use the `swapon`³¹ command.

```
[linux@basis] swapon --show
...output...
```

³¹ See the `swapon(8)` manual page

0.8 Documentation

0.8.1 Manual pages

On Linux, most documentation is located in `/usr/share/doc` and `/usr/share/man`. Debian packages that contain documentation often have the `-doc` suffix, for example `gdb-doc`.

Manual pages are one of the most useful and important sources of information and are divided into sections. The sections are listed below (as described in the `man(1)` manual page, which can be accessed using the `man man` command):

1. Executable programs and shell commands
2. System calls (functions provided by the kernel)
3. Library calls (functions within system libraries)
4. Special files (usually found in `/dev`)
5. File formats and conventions e.g. `/etc/passwd`
6. Games
7. Macro packages and conventions e.g. `man(7)`, `groff(7)`
8. Commands for system administration (usually for root only)
9. Kernel Routines [Non-Standard]

To read the manual page of the `ls` command

```
[linux@basis] man ls
```

To read the manual page of the `read` system call in the section 2

```
[linux@basis] man -s 2 read
[linux@basis] man s 2 read
[linux@basis] man 2 read
```

To view the manual page of the `man` command

```
[linux@basis] man man
```

To view the manual page of the `proc` filesystem

```
[linux@basis] man 5 proc
```

On Debian-based systems the manual pages for the commands are usually installed automatically. However, we can install them via `apt`.³²

```
[linux@basis] sudo apt-get manpages
```

To install the manual pages related to the C library and the Linux API, the `manpages-dev` package must be installed.

³² The packages of the manual pages are available in several languages (`manpages-de`, `manpages-fr`, etc..)


```
[linux@basis] sudo apt install manpages-dev  
...output...
```

To install the POSIX standard manual pages the packages are: `manpages-posix` and `manpages-posix-dev`

```
[linux@basis] sudo apt install manpages-posix manpages-posix-dev  
...output...
```

To read the manual page of the `/etc/passwd` file

```
[linux@basis] man 5 passwd
```

To read the manual page of the `passwd` command

```
[linux@basis] man passwd  
[linux@basis] man 1 passwd  
[linux@basis] man -s 1 passwd
```

To read the manual page of the `/dev/tty` terminal file

```
[linux@basis] man 4 tty
```

The manual page files are located in `/usr/share/man`. This directory contains subdirectories representing the different sections (e.g., `man1`, `man2`, etc.) as well as directories for different languages of the manual pages.

```
[linux@basis] cd /usr/share/man  
[linux@basis] ls  
ca  el  fr      hr  ja    man3  man7  pl      ru  sv    zh_CN  
cs  es  fr.ISO8859-1 hu  ko    man4  man8  pt      sk  tr    zh_TW  
da  et  fr.UTF-8   id  man1  man5  man9  pt_BR  sl    uk  
de  fi  gl      it  man2  man6  nl     ro     sr    zh
```

The files inside these subdirectories are compressed using `gzip`. For example, the manual page for the `ls` command is stored in the file `ls.1.gz`.

```
[linux@basis] ls ls.1.gz  
ls.1.gz
```

If we want to read the content we need to decompress it³³

```
[linux@basis] sudo gunzip ls.1.gz
[linux@basis] ls ls.1
ls.1
```

The file is in the `troff`³⁴ format, therefore, to read it need to use the `man` tool

```
[linux@basis] man ./ls.1
...output...
```

Another way is to use the `groff` utility, which can read the GNU `roff` format

```
[linux@basis] sudo apt install groff
...output...

[linux@basis] groff -man -T utf8 ls.1 | less -R:ls.1
...output...
```

If we want to obtain a PDF version of the manual page, we can execute the following command³⁵

```
[linux@basis] man -l -Tpdf ls.1 > ~/ls.pdf
```

We can convert the manual page into a PDF also without unzipping the file

```
[linux@basis] man -l -Tpdf ls.1 > ~/ls.pdf
```

HelpMan is a graphical user interface that allows users to select a section in order to browse and find manual pages.

We can install it using `apt`

```
[linux@basis] sudo apt install helpman
...output...
```

³³ To unzip the files in the `/usr/share/man` we need to use `sudo`

³⁴ `troff` is a program to format documents created with `roff`, which is a system used by GNU to deal with documents and typesetting, See the manual page `roff(7)`, `troff(1)`, `groff(1)`

³⁵ This command creates a PDF file inside the user home folder

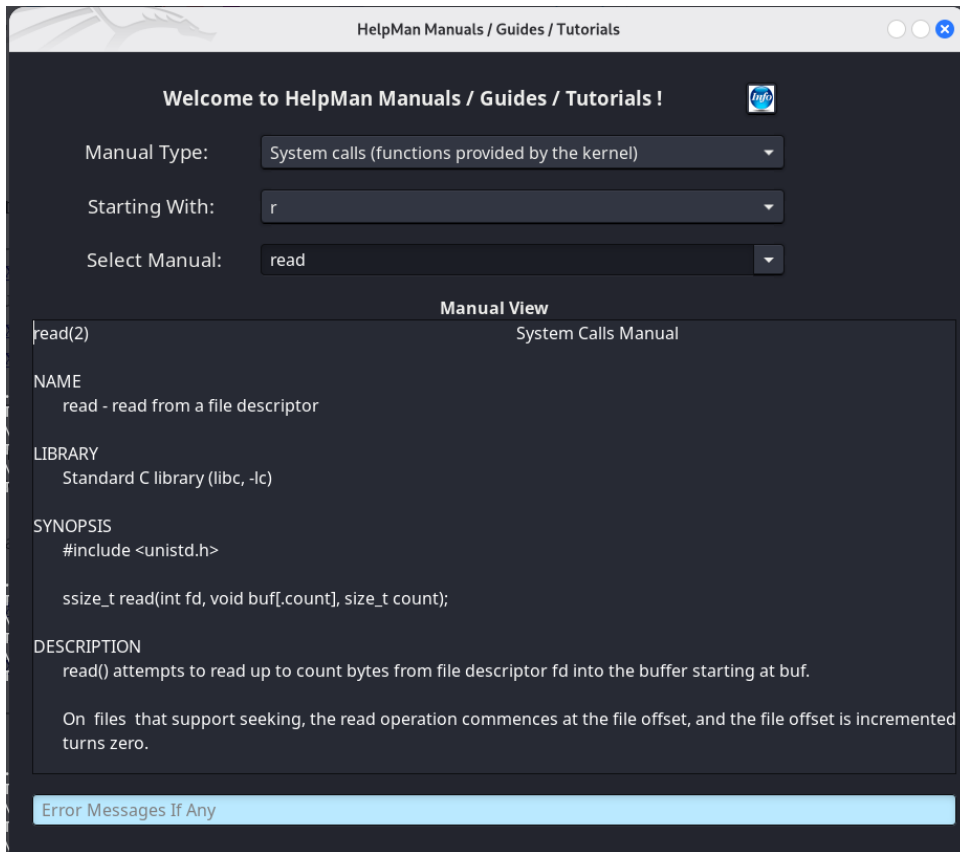


Fig 4. HelpMan

0.8.2 `what is`

`what is`³⁶ is a command-line utility that displays a short description of a command taken from its manual page.

```
[linux@basis] whatis ls
ls (1)          - list directory contents
ls (1posix)     - list directory contents
```

The number in parentheses corresponds to the manual section in which the program's manual page is found.

0.8.3 `info`

The `info`³⁷ program is a navigable documentation viewer.

```
[linux@basis] info ls
terminal screen of info...
```

0.8.4 `apropos`

The `apropos`³⁸ program takes a string as input and displays all manual pages related to that string.

```
[linux@basis] apropos bash
bash (1)          - GNU Bourne-Again SHell
bash-builtins (7) - bash built-in commands, see bash(1)
bashbug (1)       - report a bug in bash
builtins (7)      - bash built-in commands, see bash(1)
```

0.8.5 The *glibc* documentation

The *glibc* documentation can be installed using `apt`.

```
[linux@basis] sudo apt install glibc-doc glibc-doc-reference
...output...
```

Once installed, the documentation files are located in the directory

```
/usr/share/doc/glibc-doc-reference
```

To read the documentation for a header file, you can pass the header file name to `man`

```
[linux@basis] man stdio.h
```

³⁶ See the `what is(1)` manual page

³⁷ See the `info(1)` manual page

³⁸ See the `apropos(1)` manual page

0.9 The source code

The Linux kernel code is downloadable from www.kernel.org or on Debian-based systems it is installable via `apt`.

```
[linux@basis] sudo apt install linux-source
...output...
```

Once installed, it is located in the `/usr/src` directory in compressed format, so it must be decompressed and extracted.

```
[linux@basis] cd /usr/src
[linux@basis] sudo unxz linux-source-5.16.tar.xz
[linux@basis] sudo tar xvf linux-source-5.16.tar
...output...
```

Once extracted, the kernel source code can be found in the newly created directory `linux-source-5.16`.

```
[linux@basis] cd linux-source-5.16
[linux@basis] ls
arch      CREDITS      fs           Kbuild      LICENSES    net          security    virt
block     crypto       include     Kconfig     MAINTAINERS README      sound
certs     Documentation init         kernel      Makefile    samples     tools
COPYING   drivers      ipc         lib          mm          scripts     usr
```

Most Linux distributions provide kernel header files in a separate package that can be installed on Debian-based systems via `apt`. You can use the package manager to find the package corresponding to your kernel version.³⁹

```
[linux@basis] sudo apt-cache search linux-headers
linux-headers-5.16.0-kali7-amd64 - Header files for Linux 5.16.0-kali7-amd64
linux-headers-5.16.0-kali7-cloud-amd64 - Header files for Linux 5.16.0-kali7-cloud-amd64
linux-headers-5.16.0-kali7-common - Common header files for Linux 5.16.0-kali7
...output...

[linux@basis] sudo apt install linux-headers-5.16.0-kali7-amd64
```

The package containing the headers is downloaded into the directory `/usr/src`.

As for the source code of software used in Debian-based distributions, it can be downloaded from the corresponding mirrors. To obtain these sources, you need to add or uncomment the `deb-src`⁴⁰ entries in the `/etc/apt/sources.list` file.

Example of `sources.list` configured to download program sources.

³⁹ The Kernel header files are distributed by Linux distributions in a separate package from the kernel source package

⁴⁰ Visit the url <https://wiki.debian.org/it/SourcesList>

```
deb http://http.kali.org/kali kali-rolling main contrib non-free
deb-src http://http.kali.org/kali kali-rolling main contrib non-free
```

The first line represents the mirror for downloading programs, the second line the mirror for downloading sources.

Once mirrors have been configured, the source packages can be installed

```
[linux@basis] sudo apt-get update
...output...

[linux@basis] sudo apt-get source bash
...output...
```

Packages are downloaded in compressed format in the current directory

```
[linux@basis] ls
bash-5.1  bash_5.1-6.debian.tar.xz  bash_5.1-6.dsc  bash_5.1.orig.tar.xz
```

In the case of source packages, apt downloads the files and then extracts their contents. The directory contains a folder with the source code (bash-5.1), a Debian-specific package for your operating system (bash_5.1-6.debian.tar.xz), a file containing the package checksums and metadata (bash_5.1-6.dsc), and the archive with the original upstream source code (bash_5.1.orig.tar.xz).

The source files of the software are present in the decompressed folder.

```
[linux@basis] cd bash-5.1
[linux@basis] ~/bash-5.1] ls
ABOUT-NLS      command.h      findcmd.h      mksyntax.c     sig.h
aclocal.m4      COMPAT         flags.c        NEWS           siglist.c
alias.c         config-bot.h   flags.h        nojobs.c       siglist.h
alias.h         config.h.in    general.c      NOTES         stringlib.c
array.c         config-top.h   general.h      parser-built   subst.c
arrayfunc.c     configure      hashcmd.c     parser.h       subst.h
arrayfunc.h     configure.ac   hashcmd.h     parse.y        support
array.h         conftypes.h   hashlib.c     patchlevel.h  syntax.h
assoc.c         copy_cmd.c    hashlib.h     pathexp.c     test.c
assoc.h         COPYING       include        pathexp.h     test.h
AUTHORS         cross-build   input.c        pathnames.h.in tests
bashansi.h     CWRU          input.h        pcomplete.c   trap.c
bashhist.c     debian        INSTALL       pcomplete.h   trap.h
bashhist.h     dispose_cmd.c jobs.c         pcomplib.c   unwind_prot.c
bashintl.h     dispose_cmd.h jobs.h         po           unwind_prot.h
bashjmp.h      doc           lib           POSIX         variables.c
bashline.c     error.c       list.c        print_cmd.c   variables.h
bashline.h     error.h       locale.c     quit.h        version.c
bashtypes.h    eval.c       m4           RBASH         xmalloc.c
bracecomp.c    examples     mailcheck.c  README       xmalloc.h
braces.c       execute_cmd.c mailcheck.h   redir.c      Y2K
builtins       execute_cmd.h make_cmd.c   redir.h      y.tab.c
builtins.h     expr.c       make_cmd.h   shell.c       y.tab.h
ChangeLog      externs.h    Makefile.in  shell.h
```

CHANGES	findcmd.c	MANIFEST	sig.c
---------	-----------	----------	-------

Much software is distributed as source code and, once downloaded, must be compiled. The compilation process varies from one program to another, but there are some conventions which, when followed, simplify building and installing software in Linux environments. For example, if we want to install the latest version of `gawk`⁴¹ (a programming language designed for text processing and data extraction), we can download its source code from the official website and compile it. The complete procedure is shown below.

We download the source code using `wget`⁴²

```
[linux@basis] wget https://ftp.gnu.org/gnu/gawk/gawk-5.1.1.tar.xz
...output...

[linux@basis] ls
gawk-5.1.1.tar.xz
```

Unpacking the package and extracting its contents using `unxz`⁴³ and `tar`⁴⁴

```
[linux@basis] unxz gawk-5.1.1.tar.xz
[linux@basis] tar xvf gawk-5.1.1.tar
...output...
[linux@basis] ls
gawk-5.1.1  gawk-5.1.1.tar
```

Once the contents of the package have been extracted, it is possible to proceed with the compilation

```
[linux@basis] cd gawk-5.1.1
[linux@basis ~/gawk-5.1.1] ./configure
...output...
[linux@basis ~/gawk-5.1.1] make
...output...
[linux@basis ~/gawk-5.1.1] make install
...output...
```

The last command (`make install`) installs the compiled software. This is the standard procedure for software distributed via source code. In the source directory there is usually a `README` file. Some software provides an `INSTALL` file or a directory (`doc`) containing installation documentation. In any case, regardless of the software, it is always good practice to consult the documentation provided.

41 See the `gawk (1)` manual page

42 See the `wget (1)` manual page. It can be installed with `sudo apt install wget`

43 `unxz` is a tool to decompress archives, it belongs to the utility `xz`, see the `xz (1)` manual page

44 `tar` is the archiving utility, see the `tar (1)` manual page. Some packages are compressed via `gzip` or `bzip2` in which case the corresponding software must be used for the decompression (`gunzip`, `bunzip2`)

0.10 The Standards

Working with programming on Linux often implies programming in the C language. When programming in C, it is important to know the standards that define the language and the features that enable system-level programming. Below is a summary of the main standards with a brief description⁴⁵.

ISO C

This is the main standard of the C language adopted by the American National Standards Institute (ANSI) as ANSI C (X3.159-1989, “ANSI C”). It was later standardized by the International Organization for Standardization (ISO) as ISO/IEC 9899:1990 (“Programming Language C”). The GNU C Library is a superset of ISO C.

POSIX (Portable Operating System Interface for Unix)

POSIX defines a set of standards for maintaining compatibility between operating systems (ISO/IEC 9945). It was also published as ANSI/IEEE Std 1003 and is largely based on UNIX systems. POSIX provides a superset of ISO C functionality, adding system-level features and additional functions for low-level operating system support.

POSIX.1 (ISO/IEC 9945-1:1996), which defines the system API, is implemented by the GNU C Library.

POSIX.2, which defines shell and utilities, is also implemented by the GNU C Library.

Berkeley UNIX

The GNU C Library also includes features from non-formally standardized UNIX variants such as 4.2 BSD, 4.3 BSD, 4.4 BSD, and SunOS¹.

SVID

The System V Interface Description is a specification describing AT&T UNIX System V. The GNU C Library implements many features required by SVID that are not included in ISO C or POSIX.

XPG

The X/Open Portability Guide is a general standard published by the X/Open Company Ltd. X/Open owns the UNIX trademark. It defines requirements for systems intended to be UNIX-compatible. The GNU C Library is compatible with XPG and includes common extensions from XSI (X/Open System Interface) and X/Open UNIX extensions.

LSB

The Linux Standard Base is an ISO standard for GNU/Linux aimed at ensuring compatibility and standardization across Linux distributions. It incorporates other standards such as POSIX and SUS.

SUS

The Single UNIX Specification defines the requirements a system must meet to be certified as a UNIX system. Linux is considered a UNIX-like operating system.

⁴⁵ For more information see the *Bibliography*

0.11 The GNU General Public License

The GNU General Public License (GPL) is a free copyleft license for software. It is based on some principles concerning the freedom to use and distribute software.

From the official page (<https://www.gnu.org/licenses/quick-guide-gplv3.html>):

“Nobody should be restricted by the software they use. There are four freedoms that every user should have:”

- the freedom to use the software for any purpose
- the freedom to change the software to suit your needs
- the freedom to share the software with your friends and neighbors
- the freedom to share the changes you make

When a program offers users all these freedoms, we call it free software. Programmers can release the software under the terms of the GNU GPL in which case the software will be considered free software and will remain so in perpetuity, regardless of who modifies and/or distributes the program. This is what is called copyleft.

This paragraph provides only a brief introduction to the GPL license. For further details, the reader is encouraged to consult the official website: <https://www.gnu.org>

The author makes a final observation: the development of Linux has been strongly driven by the concept of free software. Over the past two decades, Linux has evolved to become one of the most widely used operating systems in both corporate and mobile environments, largely because its source code is freely available, modifiable, and redistributable. This has allowed developers around the world to contribute to its growth and widespread adoption.

Beyond technical and professional aspects, learning and passion for technology play a fundamental role when working with computers and software in general. These elements have contributed to a significant shift in perspective, both in the market and in the dissemination of knowledge, completely changing how technology is perceived and used over the last twenty years. What was once limited to a small group of scholars and researchers has become accessible to everyone thanks to free software.

Linux has played a major role in the spread of free software, and where students or researchers once had to rely on experts to understand specific technologies, Linux and free software have helped establish a level of independence that is essential in study, research, innovation, and personal exploration.

0.12 Theory and insights

0.12.1 Byte order

Byte order (*endianness*) refers to the convention a computer uses to store multi-byte data values in memory. There are two modes

Big endian Most significant (largest) byte first (MSB first)

Little endian The least significant (smallest) byte first (LSB first)⁴⁶

The order of the bytes varies from one architecture to another, for example Intel processors use *little endian* while IBM and Motorola use *big endian*. Many internet protocols use *big endian*, which is why this mode is also called *network byte order*. The two modes represent the order in which bytes are stored or transmitted. In the case of *little indian*, the *least significant byte* is transmitted first, i.e. the smallest, in the case of *big endian* the *most significant byte* (the largest) is transmitted first.

The following picture shows the representation of the hexadecimal number 0x0123 in both modes and the representation of the same number in both modes in binary.

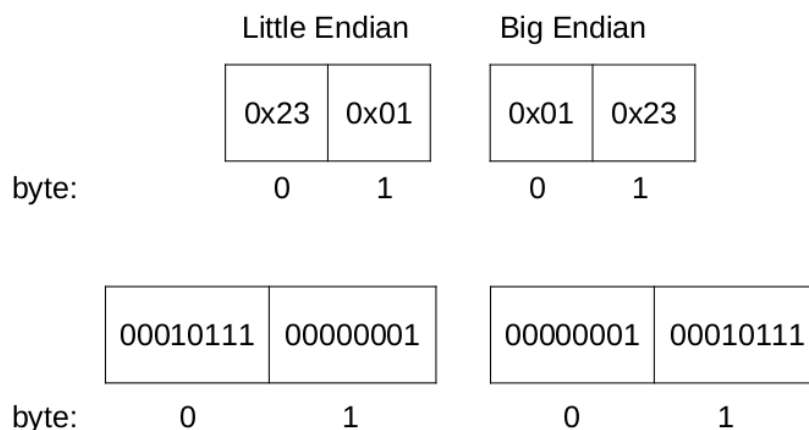


Fig 5. Byte order

The hexadecimal number 0x0123 is represented by the two bytes which are respectively the number 01 which represents the hundreds and the number 23 which represents the tens and the units. Of these two bytes the largest is byte 01 and therefore it is the most significant. In *little endian* mode byte 23 (the least significant or smallest) is transmitted first, while in *big endian* mode byte 01 (the most significant or largest) is transmitted first. The two bytes that make up the number are respectively stored in reverse from each other in the two modes.

⁴⁶ MSB stands for *Most Significant Byte*, LSB stands for *Least Significant Byte*

0.12.2 Static and dynamic linking

During the linking phase, external libraries are connected to the program so that their functions and objects can be used. With *static linking*, the code from the libraries is copied directly into the final executable file. As a result, if the program is transferred to another operating system that supports the same executable format, it can run independently, without requiring the corresponding libraries to be installed on the target system. *Statically linked programs* are usually larger because they include the library code they depend on.

With *dynamic linking*, only references to *external libraries* are stored in the executable file, while the actual linking takes place during execution, i.e. at *run time*. One advantage of *dynamic linking* is that if a library is updated, the executable usually does not need to be recompiled, since the library remains separate from the program. However, compatibility with *shared libraries* must still be maintained. In addition, only one copy of a *shared library* needs to be loaded into memory, allowing multiple programs to share it and reducing overall memory usage. On the other hand, *dynamically linked programs* may be slightly slower than *statically linked* ones due to the overhead of run-time linking.

The following picture shows the difference between *static linking* and *dynamic linking*

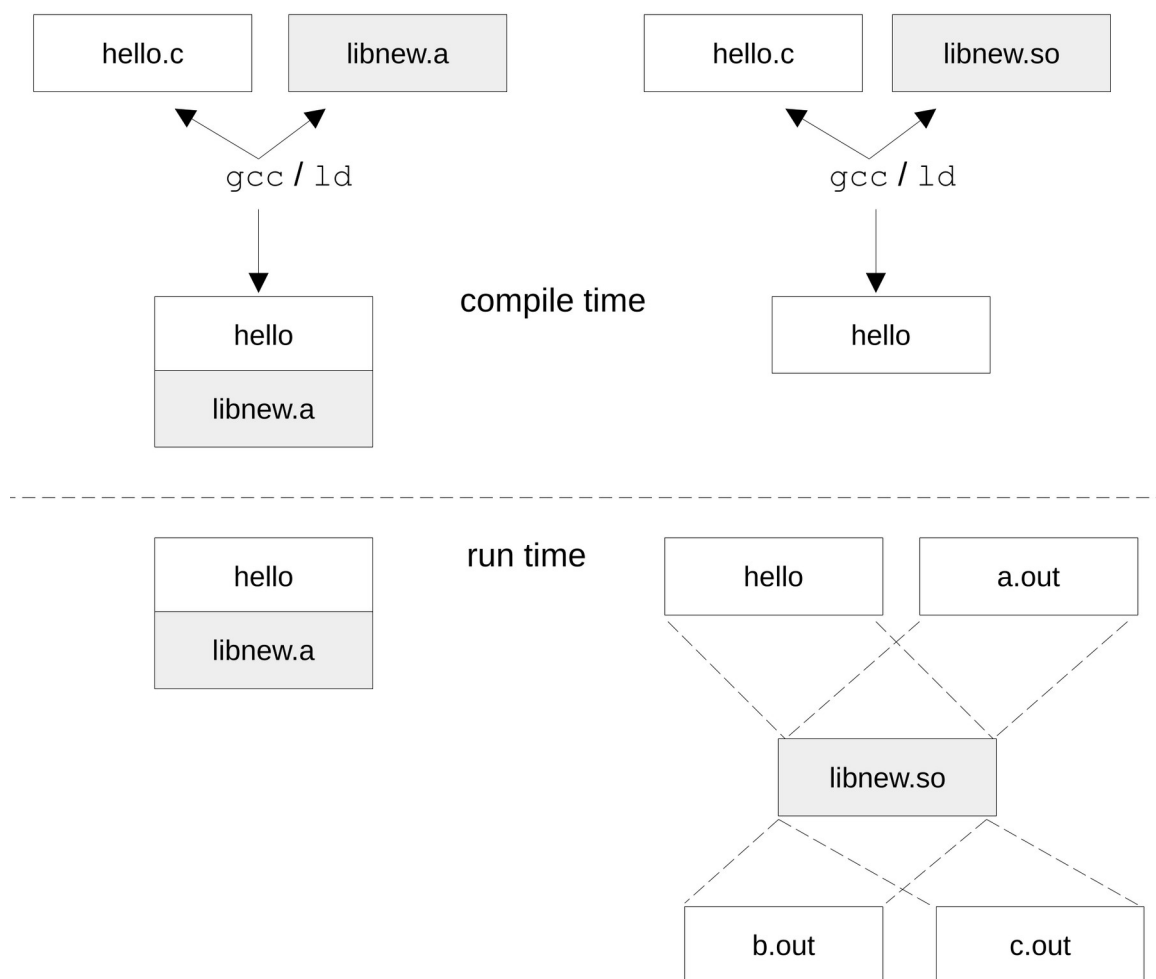


Fig 6. Static and dynamic linking

The following two examples illustrate the procedure for creating and linking a *static* and a *dynamic* library.

Static library

1. Create the file containing the library functions

```
#include <stdio.h>

void greet()
{
    printf("Hello, World!");
}
```

lib_hello.c

2. Create the header file

```
void greet();
```

lib_hello.h

3. Create the program that will use the library functions

```
#include "lib_hello.h"

void main()
{
    greet();
}
```

hello.c

4. Compile the *library file* (create the *object file*)

```
[linux@basis] gcc -c lib_hello.c -o lib_hello.o
```

5. Create the static library via `ar`⁴⁷. The `r` option tells `ar` to add the passed files to the archive, the `c` option creates the archive, the `s` option adds an index to the archive so that the contents is faster reachable

⁴⁷ `ar` is the program to create archives, see the `ar(1)` manual page

```
[linux@basis] ar rcs lib_hello.a lib_hello.o
```

`ar` has created the file `lib_hello.a` (the *static library*)

6. Compile the `hello.c` program and link to the static library

```
[linux@basis] gcc -c hello.c -o hello.o
```

```
[linux@basis] gcc -o hello hello.o -L. -l_hello
```

The `-L` option followed by the `.` tells `gcc` to look for libraries in the current directory as well as the default paths. The `-l` option followed by `_hello` specifies the name of the library to link.

7. Run the `hello` program

```
[linux@basis] ./hello  
Hello, World!
```

If we run the `file` command on the *static library* file it will display the file is an archive created with `ar`

```
[linux@basis] file lib_hello.a  
lib_hello.a: current ar archive
```

Dynamic library

To create and link a *dynamic library*, the procedure is as follows.

1. Compile the library file (`lib_hello.c`) with *Position Independent Code* via the `-fpic` option. This creates the `lib_hello.o` file

```
[linux@basis] gcc -c -fpic lib_hello.c
```

2. Create the *shared library* using the `-shared` option. This creates the `libhello.so` file.

```
[linux@basis] gcc -shared -o libhello.so lib_hello.o
```

3. Compile and link the `hello.c` program to the *shared library* `libhello.so`

```
[linux@basis] gcc -o hello hello.c -L. -lhello
```

4. Make the library available at runtime

```
[linux@basis] export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:.
```

The `export`⁴⁸ command sets the value of the shell variable and makes the variable available to the environment. The commands invoked by the shell inherit the environment and therefore the exported environment variables. The `LD_LIBRARY_PATH` variable contains the paths where the *dynamic linker* can find the *shared libraries*. The above command sets that variable to its initial value and adds to this value the path `.` (the current directory).

5. Execute `hello`.

```
[linux@basis] ./hello  
Hello, World!
```

6. Verify that the program uses the `libhello.so` shared library with `ldd`.

```
[linux@basis] ldd ./hello  
linux-vdso.so.1 (0x00007ffcd453a000)  
libhello.so (0x00007f9611a81000)  
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f9611875000)  
/lib64/ld-linux-x86-64.so.2 (0x00007f9611a8d000)
```

It is a UNIX convention that if the library file is called `libnew.so` the name of the library is *new* and therefore compilers have adopted this convention. To the `-l` option of `gcc` we just pass the library name without the `lib` prefix and the `so` extension.

If we run the `file` command on the *shared library*, it will display that the file is a *shared object*

```
[linux@basis] file libhello.so  
libhello.so: ELF 64-bit LSB shared object, x86-64, version 1 (SYSV), dynamically linked,  
BuildID[sha1]=dfdf3ade4475fa6e8f0a0829d7f0a3fdb27e8578, not stripped
```

To make the *shared library* available to all programs on the system, it must be installed in the *library path*. The copy requires *superuser permissions*.

```
[linux@basis] sudo cp libhello.so /usr/lib  
  
[linux@basis] sudo chmod 755 /usr/lib/libhello.so  
  
[linux@basis] sudo ldconfig
```

The `ldconfig`⁴⁹ program updates the *libraries cache*. This cache is used by the *dynamic linker* when executing programs and is stored in the file `/etc/ld.so.cache`.

48 See the `export(1POSIX)` manual page

49 See the `ldconfig(8)` manual page

List of tables

Tab 1. File extensions created by `-save-temps`

Tab 2. `gcc`'s options

Tab 3. `gdb` commands

List of figures

Fig 1. Compilation

Fig 2. Compilation steps with `gcc`

Fig 3. Steps of the build process with `autoconf`

Fig 4. HelpMan

Fig 5. Byte order

Fig 6. Static and dynamic linking

Bibliography

- Alejandro Colomar, Michael Kerrisk, Andries Brouwer, “GNU/Linux Programmer’s Manual”, 2020
- Bill O. Gallmeister, “POSIX.4: Programming for the Real World”, O’Reilly, 1995
- Brian W. Kernighan & Dennis M. Ritchie, “The C Programming Language”, 2ed, Prentice Hall, 1988
- Dennis Andriesse, “Practical Binary Analysis”, No Starch Press, 2019
- Donald A. Lewine, “POSIX Programmer’s Guide – Writing Portable UNIX Programs with the POSIX.1 Standard”, O’Reilly, 1994
- Intel, The Intel 64 and IA-32 Architectures Software Developer's Manual, Intel 2024
- Jack-Bennny Persson, “Linux System Programming Techniques”, Packt Publishing, 2021
- Kurt Wall, Mark Watson, Mark Whitis, “Linux Programming Unleashed”, Sams, 1999
- Michael Kerrisk, “The Linux Programming Interface”, No Starch Press, 2010
- Paul Deitel & Harvey Deitel, “C How to program”, 9ed, Pearson, 2022
- Robert Love, “Linux System Programming”, O’Reilly, 2013
- Sandra Loosemore, Richard M. Stallman, Roland McGrath, Andrew Oram, Ulrich Drepper “The GNU C Library Reference Manual” for version 2.34, Free Software Foundation
- W. Richard Stevens & Stephen A. Rago, “Advanced Programming in the UNIX Environment”, 3ed, Addison-Wesely, 2013
- International Organization for Standardization. (2011). ISO/IEC 9899:2011: Information technology - Programming languages - C

Online resources

GNU/Linux

Atomic operations library	https://en.cppreference.com/w/c/atomic.html
Autoconf	https://www.gnu.org/software/autoconf/
Automake	https://www.gnu.org/software/automake/
Bash reference manual	https://www.gnu.org/software/bash/manual/bash.html
Berkeley Packet Filter	https://www.kernel.org/doc/html/latest/bpf/index.html
Binutils	https://www.gnu.org/software/binutils/
GCC online documentation	https://gcc.gnu.org/onlinedocs/
GDB: The GNU Debugger	https://www.sourceware.org/gdb/documentation/ https://sourceware.org/gdb/current/onlinedocs/gdb.pdf
GLIBC	https://www.gnu.org/software/libc/
The GNU C Library (glibc) manual	https://sourceware.org/glibc/manual/
GNU Coreutils	https://www.gnu.org/software/coreutils/manual/
GNU General Public License	https://www.gnu.org/licenses/gpl-3.0.html
GNU Gnulib	https://www.gnu.org/software/gnulib/manual/gnulib.html
GNU Official Site	https://www.gnu.org/home.en.html
GNU Make	https://www.gnu.org/software/make/manual/
GNU Software	https://www.gnu.org/software
io_uring	https://unixism.net/loti/index.html
Libseccomp	https://github.com/seccomp/libseccomp/wiki
Libseccomp documentation	https://libseccomp.readthedocs.io/en/latest/
Liburing	https://github.com/axboe/liburing
Linux manual pages online	https://www.man7.org
Linux Kernel	https://kernel.org/
Linux Standard Base	https://refspecs.linuxbase.org/LSB_5.0.0/allspecs.shtml
Malloc	https://sourceware.org/glibc/wiki/MallocInternals
The /proc filesystem	https://www.kernel.org/doc/html/latest/filesystems/proc.html
The GNU C Reference Manual	https://www.gnu.org/software/gnu-c-manual/gnu-c-manual.html

Operating systems

Debian OS	https://www.debian.org/ https://www.debian.org/doc/devel-manuals
Debian APT User's Guide	https://www.debian.org/doc/manuals/apt-guide/index.en.html
Kali Linux	https://www.kali.org/
Linux Mint	https://linuxmint.com/
Ubuntu OS	https://ubuntu.com/
BSD Official site	https://www.bsd.org/
SunOS	https://www.oracle.com/solaris/solaris10/downloads/solaris10-get-jsp-downloads.html

Standards

ABI	https://wiki.osdev.org/System_V_ABI https://refspecs.linuxbase.org/elf/x86_64-abi-0.99.pdf https://refspecs.linuxbase.org/elf/x86_64-abi-0.21.pdf
C Language Standard	http://www.open-std.org/jtc1/sc22/wg14/
ELF Standard	https://refspecs.linuxfoundation.org/elf/elf.pdf
Linux Standard Base Specification	https://refspecs.linuxfoundation.org/lsb.shtml
System V SCO Dev Specification	http://www.sco.com/developers/devspecs/
The Single UNIX Specification	https://unix.org/what_is_unix/single_unix_specification.html
POSIX Standard	https://pubs.opengroup.org/onlinepubs/9699919799.2018edition/

X/Open Portability Guide
List of the Unicode symbols

<http://archive.opengroup.org/publications/archive/CDROM/g501.pdf>
<https://symbl.cc/en/unicode-table/>